

FLIGHT CENTER Reservation System — Detailed Technical Documentation

Version 2.0 — Extended Edition

Table of Contents

- Executive Summary
- Project Background and Problem Statement
- Goals and Objectives
- Scope and Boundaries
- Stakeholders
- Functional Requirements
- Non-Functional Requirements
- System Architecture
- Technology Stack
- System Modules (Detailed)
- Database Schema
- API Design
- User Interface Overview
- Workflow: End-to-End Booking Process
- Security Architecture
- Testing Strategy
- Deployment Plan
- Maintenance and Support
- Risk Analysis
- Limitations
- Future Roadmap
- Cost Estimation
- Glossary
- Conclusion
- References

1. Executive Summary

Flight Center is developing a modern, web-based airline reservation platform intended to replace legacy manual booking processes with a centralized digital system. This document expands on the core project documentation by providing deeper technical detail across architecture, database design, workflows, security, and deployment considerations, intended for engineering and QA teams rather than a general audience.

The platform will allow customers to search flights, hold and pay for seats, manage bookings, and receive automated notifications, while giving Flight Center staff tools to manage inventory, pricing, and customer service operations from a single administrative console.

2. Project Background and Problem Statement

Flight Center currently relies on a mix of spreadsheets, phone bookings, and a partially automated legacy tool for managing reservations. This creates duplicate bookings, delayed confirmations, inconsistent pricing across sales channels, and limited visibility into real-time seat inventory.

The proposed system addresses these issues by consolidating booking, payment, and inventory management into one platform with a single source of truth for flight and seat data, accessible to both customers and staff in real time.

3. Goals and Objectives

- Reduce booking processing time from an average of 8 minutes (manual) to under 90 seconds (self-service).
- Eliminate double-booking of seats through atomic seat-locking during checkout.
- Provide a unified admin console for flight scheduling, pricing, and reporting.
- Support multiple payment gateways and currencies.
- Achieve 99.5% platform uptime during business hours.
- Enable staff to issue refunds and manage cancellations within defined policy rules without engineering involvement.

4. Scope and Boundaries

In scope: flight search, seat selection, booking, payment processing, e-ticket generation, cancellation and refund handling, admin flight management, and basic customer support tooling.

Out of scope for this release: baggage tracking integration, loyalty/frequent-flyer point systems, third-party travel agency APIs, and multi-language localization. These are addressed in the Future Roadmap section.

5. Stakeholders

Stakeholder	Interest / Role
Customers	Search and book flights, manage their own reservations
Booking Agents	Create and modify bookings on behalf of customers
Flight Operations Staff	Manage flight schedules, aircraft assignment, seat maps
Finance Team	Reconcile payments, process refunds, generate revenue reports
IT/Engineering	Build, deploy, and maintain the platform
Executive Sponsors	Approve budget and monitor project KPIs

6. Functional Requirements

6.1 Customer-Facing Requirements

- FR-01: Users shall be able to register using email or phone number with OTP verification.
- FR-02: Users shall be able to search flights by origin, destination, date, and passenger count.
- FR-03: Users shall be able to view seat maps and select specific seats where available.
- FR-04: Users shall be able to pay via card, digital wallet, or bank transfer.
- FR-05: Users shall receive an e-ticket and booking confirmation via email/SMS within 60 seconds of successful payment.
- FR-06: Users shall be able to cancel or modify a booking subject to fare rules.

6.2 Admin-Facing Requirements

- FR-07: Admins shall be able to create, update, and cancel flight schedules.
- FR-08: Admins shall be able to adjust fare classes and pricing rules per route and date range.
- FR-09: Admins shall be able to view real-time seat occupancy per flight.
- FR-10: Admins shall be able to issue full or partial refunds with an audit trail.
- FR-11: Admins shall be able to export booking and revenue reports in CSV/PDF format.

7. Non-Functional Requirements

Category	Requirement
Performance	Search results returned within 2 seconds for 95% of queries
Scalability	Support 10,000 concurrent users during peak booking windows
Availability	99.5% uptime during business hours; 99% overall
Security	All traffic over TLS 1.2+; PCI-DSS aligned payment handling
Usability	Booking flow completable in 5 steps or fewer
Auditability	All admin actions logged with timestamp and user ID

8. System Architecture

The system follows a three-tier architecture: a React-based single-page front end, a REST API layer built on Node.js/Express (or Django, depending on final stack decision), and a relational database (PostgreSQL) for transactional data, supplemented by a caching layer (Redis) for seat-hold locks and session data.

A message queue (e.g., RabbitMQ) decouples booking confirmation from downstream tasks such as email/SMS notifications and invoice generation, so a slow notification provider cannot block the checkout flow.

High-level component list:

- Web Client (React/Angular SPA)
- API Gateway / Load Balancer
- Application Server(s) — booking, pricing, and auth services
- Cache Layer (Redis) — seat locks, session tokens
- Primary Database (PostgreSQL) — flights, users, reservations, payments
- Message Queue — asynchronous notification and reporting jobs
- Payment Gateway Integration (Stripe/PayPal/local processors)

9. Technology Stack

Layer	Technology Options
Frontend	React or Angular, TypeScript, Tailwind CSS
Backend	Node.js (Express/NestJS) or Python (Django/Flask)
Database	PostgreSQL (primary), Redis (cache/session)
Messaging	RabbitMQ or AWS SQS
Hosting	AWS/Azure/GCP, containerized via Docker + Kubernetes

CI/CD	GitHub Actions / GitLab CI
Monitoring	Prometheus + Grafana, Sentry for error tracking

10. System Modules (Detailed)

10.1 User Module

- Registration with email/phone verification and password strength enforcement.
- Login with optional two-factor authentication (TOTP or SMS OTP).
- Profile management: contact details, saved passengers, payment methods on file.
- Password reset via time-limited secure token.

10.2 Admin Module

- Flight creation and scheduling, including recurring flight templates.
- Aircraft and seat map configuration per flight.
- Fare class and pricing rule management, including seasonal pricing.
- User account management, including suspension for fraud review.
- Dashboard with real-time KPIs: bookings today, revenue, load factor per route.

10.3 Reservation Module

- Real-time seat availability check with temporary seat hold (default 10-minute lock).
- Multi-passenger booking in a single transaction.
- Booking modification (date change, seat change) subject to fare rules.
- Cancellation workflow with automated refund calculation based on fare class.

10.4 Payment Module

- Integration with multiple payment gateways with a common internal interface.
- Idempotent payment processing to prevent duplicate charges on retry.
- Automatic invoice and receipt PDF generation.
- Refund processing with reconciliation against the original transaction.

10.5 Notification Module (New)

- Email and SMS confirmations for booking, payment, cancellation, and schedule changes.
- Push notifications for mobile app users regarding gate changes or delays.

- Configurable notification templates managed by admin staff.

10.6 Reporting Module (New)

- Daily and monthly revenue reports by route and fare class.
- Load factor and occupancy analytics per flight.
- Exportable CSV/PDF reports for finance and operations teams.

11. Database Schema

The schema below expands on the basic table list with data types and relationships, intended to guide actual implementation.

Table	Key Columns
Users	UserID (PK), Name, Email, Phone, PasswordHash, Role, CreatedAt
Flights	FlightID (PK), FlightNumber, Source, Destination, DepartureTime, ArrivalTime, AircraftID
Aircraft	AircraftID (PK), Model, TotalSeats, SeatMapConfig
FareClasses	FareClassID (PK), FlightID (FK), ClassName, Price, SeatsAvailable
Reservations	ReservationID (PK), UserID (FK), FlightID (FK), FareClassID (FK), Status, CreatedAt
Passengers	PassengerID (PK), ReservationID (FK), Name, PassportNumber, SeatNumber
Payments	PaymentID (PK), ReservationID (FK), Amount, Currency, Status, GatewayRef
Notifications	NotificationID (PK), UserID (FK), Type, Channel, SentAt

Foreign key relationships enforce referential integrity: a Reservation cannot exist without a valid User and Flight, and a Payment must reference a valid Reservation. Cascading deletes are intentionally disabled on Reservations to preserve historical audit records.

12. API Design

Representative REST endpoints:

Endpoint	Method	Description
/api/flights/search	GET	Search flights by origin, destination, date
/api/reservations	POST	Create a new reservation (holds seats)
/api/reservations/:id	GET	Retrieve reservation details

/api/reservations/:id/cancel	POST	Cancel a reservation
/api/payments	POST	Process payment for a reservation
/api/admin/flights	POST	Create a new flight schedule
/api/admin/reports/revenue	GET	Retrieve revenue report

13. User Interface Overview

The customer-facing UI consists of a search landing page, a flight results list with filters (price, duration, stops), a seat-selection screen, a checkout/payment screen, and a booking confirmation/e-ticket page. The admin console includes a dashboard, flight scheduling calendar, pricing rule editor, and a reports section.

14. Workflow: End-to-End Booking Process

- Step 1: Customer searches for flights by route and date.
- Step 2: System displays available flights with fare classes and prices.
- Step 3: Customer selects a flight and fare class; seats are temporarily held for 10 minutes.
- Step 4: Customer enters passenger details and selects seats.
- Step 5: Customer proceeds to payment; the payment gateway processes the transaction.
- Step 6: On successful payment, the reservation status changes to Confirmed and seat holds convert to permanent allocations.
- Step 7: An e-ticket and confirmation notification are generated and sent asynchronously via the message queue.
- Step 8: If payment fails or the hold expires, seats are released back to inventory automatically.

15. Security Architecture

- Passwords stored using a salted hashing algorithm (bcrypt or Argon2), never in plain text.
- Two-factor authentication available for both customer and admin accounts.
- All API traffic encrypted in transit via TLS 1.2 or higher.
- Role-based access control (RBAC) restricting admin functions by staff role.
- Payment card data handled through a PCI-DSS compliant gateway; card numbers never stored on internal servers.
- Rate limiting and CAPTCHA on login and registration endpoints to mitigate brute-force and bot activity.
- Full audit logging of admin actions, including refunds and schedule changes.

16. Testing Strategy

16.1 Test Levels

- Unit testing for individual service functions (pricing calculation, seat lock logic).
- Integration testing across booking, payment, and notification services.
- End-to-end testing simulating full customer booking journeys.
- Load testing to validate performance under 10,000 concurrent users.
- Security testing including penetration testing before production launch.

16.2 Sample Test Cases

Test ID	Scenario	Expected Result
TC-01	Search flights with valid route and date	Matching flights returned within 2 seconds
TC-02	Attempt to book last available seat twice simultaneously	Only one booking succeeds; second is rejected
TC-03	Payment fails mid-transaction	Reservation reverts to Pending/Released, no charge applied
TC-04	Cancel a booking within refund window	Refund calculated per fare rules and processed
TC-05	Admin edits pricing for a route	New price reflected in subsequent searches only

17. Deployment Plan

The application will be containerized using Docker and orchestrated with Kubernetes, enabling horizontal scaling of the application tier during peak booking periods. A blue-green deployment strategy will be used to minimize downtime during releases, with automated rollback if health checks fail post-deployment.

18. Maintenance and Support

- 24/7 monitoring with alerting for error rate spikes and latency degradation.
- Scheduled database backups every 6 hours with 30-day retention.
- Monthly patching cycle for dependencies and security updates.
- Tiered support: Level 1 (customer support), Level 2 (operations), Level 3 (engineering on-call).

19. Risk Analysis

Risk	Impact	Mitigation
Payment gateway outage	Bookings cannot be completed	Support a secondary backup gateway
Double booking under high concurrency	Overbooked flights, customer dissatisfaction	Atomic seat-locking with Redis
Data breach of customer information	Legal and reputational damage	Encryption, RBAC, regular security audits
Third-party API rate limits	Notification delays	Queue-based retry with backoff

20. Limitations

- No support for multi-leg or codeshare itineraries in the initial release.
- No built-in loyalty or frequent-flyer program.
- Limited to a single default language (English) at launch.
- No offline/native mobile app in the first release; web is mobile-responsive only.

21. Future Roadmap

- Loyalty program with tiered frequent-flyer rewards.
- Native iOS and Android apps with offline boarding pass storage.
- Multi-language and multi-currency support.
- Integration with third-party travel agencies and global distribution systems (GDS).
- AI-based dynamic pricing recommendations for revenue management.

22. Cost Estimation

Category	Estimated Cost (USD)
Development (6 months, 5-person team)	\$150,000 – \$220,000
Cloud infrastructure (annual)	\$18,000 – \$30,000
Third-party payment gateway fees	2.5% – 3% per transaction
Ongoing maintenance (annual)	\$25,000 – \$40,000

23. Glossary

Term	Definition
PNR	Passenger Name Record — unique booking reference code
Fare Class	A pricing/service tier for a flight (e.g., Economy, Business)
Seat Hold	Temporary reservation of a seat pending payment completion
Load Factor	Percentage of available seats sold on a given flight
RBAC	Role-Based Access Control

24. Conclusion

This extended documentation provides the additional architectural, security, and operational detail needed to move the Flight Center reservation platform from concept to implementation. Combined with the original overview document, it gives both business stakeholders and engineering teams a shared reference for scope, design, and delivery expectations.

25. References

- PCI Security Standards Council — Payment Card Industry Data Security Standard documentation.
- PostgreSQL Official Documentation.
- OWASP Top Ten Web Application Security Risks.
- Kubernetes Official Documentation.
- IATA guidelines on passenger reservation systems (general industry reference).